

GymLog PWA: Offline-First Workout Tracker

Architecture Blueprint, Data Strategy, and Mobile Implementation Roadmap

TARGET PLATFORM	DELIVERY TEAM	INFRASTRUCTURE COST	STATUS
Mobile-First PWA	Antigravity Eng	\$0 / Infinite Free Tier	Ready for Implementation

Core Product Mandate & Operating Constraints

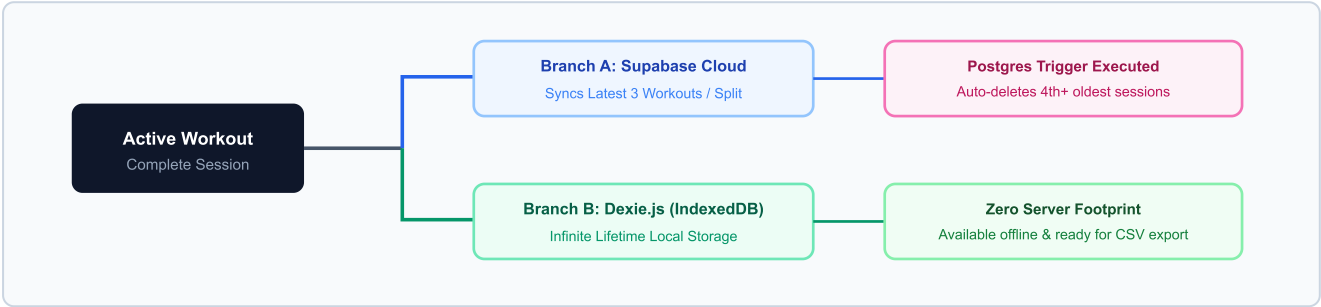
GymLog is a zero-latency, mobile-first workout tracking application purpose-built for poor cellular environments (gym basements and shielded zones). The architecture strictly maintains zero cloud operating costs through a **Dual-Tier Hybrid Storage Model**: the cloud database strictly retains only the **last 3 workouts per split** via automated PostgreSQL triggers, while the user's mobile device stores an **infinite lifetime archive** via IndexedDB.

1. TECHNICAL STACK & INFRASTRUCTURE ARCHITECTURE

Layer	Technology Selection	Strategic Rationale	Cost Ceiling
Client Shell	React 18 (Vite) + TypeScript	Sub-second load times, strict type safety, modular component tree.	Open Source (\$0)
UI / Styling	Tailwind CSS + Lucide Icons	Utility-first mobile responsive tokens; native dark mode by default.	Open Source (\$0)
PWA Runtime	Vite PWA Plugin (Workbox)	Full offline service-worker caching, asset precaching, install prompt.	Open Source (\$0)
Client Archive	Dexie.js (IndexedDB)	Unlimited lifetime workout history stored locally on the member's device.	100% Client-Side (\$0)
Cloud DB & Auth	Supabase (PostgreSQL + Auth)	Row Level Security (RLS), email/password auth, auto-pruning triggers.	50,000 MAU Free
Edge Deployment	Cloudflare Pages or Vercel	Global edge delivery, automated SSL, continuous Git deployment.	Unlimited Bandwidth (\$0)

2. DUAL-LAYER STORAGE ARCHITECTURE FLOW

When a member finishes logging a workout, the app executes a dual-write process that guarantees complete data security, permanent record keeping, and absolute cloud zero-bloat:



3. DATABASE SCHEMA, AUTOMATIONS & AUTHENTICATION

The backend relies on Supabase PostgreSQL with declarative Row Level Security (RLS) policies. Every query is scoped to the authenticated session context (``auth.uid()``).

```
-- 1. Profiles Table (Holds identity, display preferences, and unit selection) CREATE TABLE
public.profiles ( id UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY, full_name TEXT NOT NULL
DEFAULT 'Gym Member', unit_preference TEXT NOT NULL DEFAULT 'kg' CHECK (unit_preference IN ('kg', 'lbs')),
created_at TIMESTAMPTZ DEFAULT TIMEZONE('utc', NOW()) NOT NULL ); -- 2. Exercises Table (Shared global
library + member-created custom exercises) CREATE TABLE public.exercises ( id UUID DEFAULT
gen_random_uuid() PRIMARY KEY, user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE, -- NULL = System
Default, UUID = Custom name TEXT NOT NULL, split_type TEXT NOT NULL, -- e.g. 'Push', 'Pull', 'Legs',
'Arms', 'Core' machine_settings TEXT DEFAULT '', -- Micro-notes: "Seat pin #4, handle notch 2" created_at
TIMESTAMPTZ DEFAULT TIMEZONE('utc', NOW()) NOT NULL ); -- 3. Workouts Table (Scoped to split_type for
precise auto-pruning) CREATE TABLE public.workouts ( id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL, title TEXT NOT NULL, split_type TEXT
NOT NULL, started_at TIMESTAMPTZ DEFAULT TIMEZONE('utc', NOW()) NOT NULL, completed_at TIMESTAMPTZ, notes
TEXT ); -- 4. Workout Sets Table (Cascades deletion automatically when workout is pruned) CREATE TABLE
public.workout_sets ( id UUID DEFAULT gen_random_uuid() PRIMARY KEY, workout_id UUID REFERENCES
public.workouts(id) ON DELETE CASCADE NOT NULL, exercise_id UUID REFERENCES public.exercises(id) ON DELETE
RESTRICT NOT NULL, set_number INT NOT NULL, weight NUMERIC(6,2) NOT NULL, reps INT NOT NULL, is_completed
BOOLEAN DEFAULT false NOT NULL, created_at TIMESTAMPTZ DEFAULT TIMEZONE('utc', NOW()) NOT NULL );
```

3.1 Zero-Cost Cloud Pruning Engine (Automated Trigger)

To prevent database growth and stay within the free tier indefinitely, PostgreSQL autonomously manages row counts. Whenever a workout is completed and inserted, rows exceeding the 3 most recent sessions for that specific split are deleted:

```
CREATE OR REPLACE FUNCTION public.prune_excess_workouts() RETURNS TRIGGER AS $$ BEGIN DELETE FROM
public.workouts WHERE id IN ( SELECT id FROM public.workouts WHERE user_id = NEW.user_id AND split_type =
NEW.split_type ORDER BY started_at DESC OFFSET 3 ); RETURN NEW; END; $$ LANGUAGE plpgsql SECURITY DEFINER;
CREATE TRIGGER trigger_prune_excess_workouts AFTER INSERT ON public.workouts FOR EACH ROW EXECUTE FUNCTION
public.prune_excess_workouts();
```

3.2 Email + Password Authentication & Profile Sync Pipeline

The app implements standard email/password authentication with automated record provisioning:

- **Registration & Automated Profile Creation:** Upon successful submission to ``auth.users``, a database trigger (``on_auth_user_created``) inserts a corresponding row into ``public.profiles`` using registration metadata (``full_name``, ``unit_preference``).
- **Session Persistence:** Supabase client caches tokens in local device storage, maintaining continuous sign-in even across offline periods.
- **Account Security:** Includes email password-recovery routes (``supabase.auth.resetPasswordForEmail``) and in-app password modification modals.

4. CORE MOBILE UX & GYM-FLOOR FEATURE SPECIFICATIONS

Every interaction is designed to eliminate logging friction on the gym floor while holding dumbbells, wearing straps, or resting between heavy sets.

"Clone Last Workout" Quickstart 1-Tap

Tapping a split (e.g., "Leg Day") shows a primary action to clone the most recent session. It pulls all exercises from the last workout and sets previous weights and rep targets as ghost-text placeholders in input fields.

Rapid-Increment Steppers Touch UI

Eliminates manual typing. Inputs display large numeric keypads (`inputmode="decimal"`) paired with one-tap pill chips: +1.25, +2.5, +5, +10 kg/lbs and +1 / -1 reps.

Lock-Screen Rest Timer Web APIs

Uses **Screen Wake Lock API** to keep the screen active during workouts. Rest intervals trigger distinct haptic feedback (`navigator.vibrate`) and synthesized Web Audio cues that transmit directly to Bluetooth headphones.

Equipment Micro-Notes Auto-Save

A compact inline note under each exercise header (e.g., "Seat height 4, pin on hole 3"). Saves directly to the exercise record on blur and surfaces automatically in all subsequent workouts.

Local Infinite Archive (Dexie.js) IndexedDB

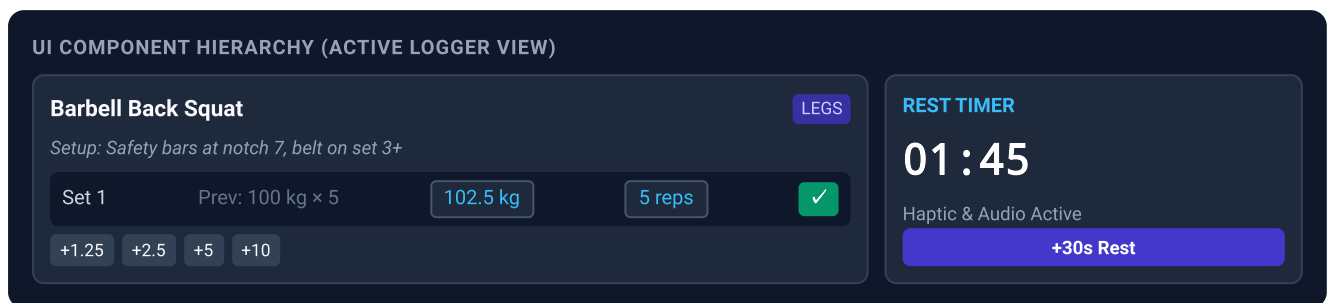
While PostgreSQL prunes records beyond 3 workouts, Dexie.js captures an identical copy client-side. The user enjoys months or years of historical training records locally without generating cloud storage costs.

1-Tap CSV / JSON Data Export Data Freedom

Allows members to export their complete IndexedDB workout archive into structured .csv or .json files at any time directly through the browser.

4.1 Mobile Active Logger Interface Layout

The active logger component architecture must conform to the following structural layout:



5. PROFILE MANAGEMENT & LOCAL STORAGE CONTROLS

The Profile screen serves as the configuration center for user identity, measurement unit standards, and client-side database management:

Module Section	UI / Functional Capability	Technical Execution
Identity & Security	Full name display, email address, password change, account deletion.	Direct integration with Supabase Auth API & profiles update.
Unit Preference	One-tap toggle between kg and lbs .	Updates <code>unit_preference</code> ; re-renders steppers across logger.
Lifetime Metrics	Total workouts logged, current weekly streak, total volume.	Aggregated locally from IndexedDB (Dexie) without cloud overhead.
Data Ownership	"Export My History" (CSV & JSON format).	Streams local DB rows into client-side downloadable Blob.

6. IMPLEMENTATION PHASES & DELIVERY ROADMAP

The development plan is broken down into four sequential execution milestones:

Phase	Deliverables & Scope	Acceptance Criteria	Target
Phase 1: Foundation	- Vite + React + Tailwind scaffold - Supabase Auth (Email + Password) - Database tables, RLS & auto-pruning trigger	Users can sign up/in; database automatically provisions profiles and purges workouts past the 3-session cap.	Sprint 1
Phase 2: Logger Engine	- Active workout UI with exercise picker - Custom exercise creation modal - Set logging, ghost-text & steppers	Members can select exercises, add sets, adjust weights with rapid pills, and add equipment micro-notes.	Sprint 2
Phase 3: Offline & Timer	- Dexie.js local mirror setup - Screen Wake Lock & Haptic Rest Timer "Clone Last Workout" logic	Timer alerts work reliably over Bluetooth/haptics; cloning fills historical sets instantly; offline workouts save.	Sprint 3
Phase 4: Export & PWA	- Profile screen with unit toggle (kg/lbs) - CSV / JSON client-side export utility - PWA service worker offline testing & iOS QA	App installs as standalone on iOS Safari & Android; full data export runs in under 1 second.	Sprint 4

7. NON-FUNCTIONAL ENGINEERING REQUIREMENTS

- Zero-Data-Loss Architecture:** Active workout states must continuously write to browser local storage. Refreshing or switching apps must never wipe in-progress sets.
- Strict Cloud Zero-Bloat:** Cloud database storage footprint must never exceed an average of ~60 rows per user, regardless of membership longevity.
- Touch Ergonomics:** Every interactive button on the active logger must meet minimum touch dimensions of 44×44 points to prevent mis-taps on the gym floor.
- Network Resilience:** If an active network connection drops mid-workout, all API requests queue silently and sync back to Supabase once connectivity is re-established.